MICROSERVICES

Study Notes

Chapters 4 and 5

Rightsizing Your Microservices: Finding Service Boundaries | Dealing with the Data

Source: Microservices: Up and Running
Ronnie Mitra and Irakli Nadareishvili (O'Reilly)

All content in these notes is drawn only from the prescribed textbook.

Contents

4. Rightsizing Your Microservices: Finding Service Boundaries

- Why Boundaries Matter, When They Matter, and How to Find Them
- Domain-Driven Design and Microservice Boundaries
- Introduction to Event Storming
- Introducing the Universal Sizing Formula

5. Dealing with the Data

- Independent Deployability and Data Sharing
- Microservices Embed Their Data
- Event Sourcing and CQRS
- Event Sourcing and CQRS Beyond Microservices

Quick Revision Sheet

- Key terms
- Key decisions
- Likely exam questions

4. Rightsizing Your Microservices: Finding Service Boundaries

Identifying proper microservice boundaries is one of the most challenging parts of building a microservices system. Drawing boundaries incorrectly can significantly diminish the benefits of microservices or even derail the effort. The most frequent question practitioners ask is: **how can a bigger application be properly sliced into a collection of microservices?**

The chapter covers three things: **Domain-Driven Design (DDD)** as the leading methodology for analyzing and decomposing large domains; **Event Storming**, a more efficient way of doing the domain analysis; and the **Universal Sizing Formula**, guidance for sizing microservices.

4.1 Why Boundaries Matter, When They Matter, and How to Find Them

The "micro" in microservices raises the question of how small services should be. The book examines simplistic answers and shows why they fail:

Simplistic approach	Why it fails
Measure by physical size or lines of code (SLOC)	SLOC has been used as a measure of effort or complexity but is widely acknowledged to be a poor measure of true size or complexity, and is easily manipulated. A fixed cap (e.g., 500 lines) is arbitrary.
Draw boundaries at functional edges (e.g., each serverless function such as an AWS Lambda is a microservice)	Tempting, but has serious problems, listed below.

Two major problems with treating functions as microservices:

- **Drawing boundaries based on technical needs is an anti-pattern.** Per Lewis and Fowler, microservices should be "organized around business capabilities", not technical needs. Parnas (1972) recommends decomposing by modular encapsulation of design changes over time. Neither aligns strongly with the boundaries of serverless functions.
- **Too much granularity, too soon.** An explosive level of granularity early in the project life cycle introduces crushing complexity that can stop the effort before it succeeds.

Since the primary goal of microservices is to **minimize coordination costs** (Chapter 1), services must be sized to minimize coordination between teams. Splitting code into functions does not necessarily do that, so you end up with incorrectly sized services.

Premature optimization: early adopters such as Netflix, SoundCloud and Amazon eventually had many microservices, but they did not start with hundreds. A large number was what they optimized toward over years, after reaching the operational maturity to handle the complexity.

> **Avoid creating too many microservices too early**
>
> Sizing is a **journey that unfolds in time**. Whether greenfield or decomposing a monolith, **start with only a handful of services and slowly increase the number**. If some services are initially larger than their target state, that is fine; they can be split later.

4.2 Domain-Driven Design and Microservice Boundaries

Sam Newman's three ground rules for boundaries

Property	Meaning
Loosely coupled	Services are fairly unaware of and independent from each other, so a change in one does not ripple into others. Limit the types of runtime calls between services, since chatty communication also leads to tight coupling.
Highly cohesive	Features inside a service are highly related; unrelated features live elsewhere. A logical change can be made in one place, minimizing time to release. Otherwise many services must be released together, needing heavy coordination.
Aligned with business capabilities	Requests for change are driven by business needs. Detailed canonical data models change often and cause rework; business capabilities are more durable (an accounting module will always provide its capabilities regardless of internal changes).

These principles are useful but high-level and aspirational; they do not give day-to-day sizing guidance. Many practitioners therefore turned to Domain-Driven Design.

Domain-Driven Design (DDD)

DDD predates microservices. It was introduced by **Eric Evans in 2003** in *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Its main premise is that, for complex systems, we should **avoid seeking a single unified domain model**. Multiple models coexist, and different models apply in different contexts.

> **Bounded context**
> Evans: a Bounded Context defines the **range of applicability of each model**. Bounded contexts let different parts of a larger system be implemented and run without corrupting the independent domain models in that system.

Ubiquitous Language

A well-defined domain model provides a **common vocabulary** of defined terms that subject-matter experts and engineers develop together, balancing business requirements and implementation considerations. This shared vocabulary is the **Ubiquitous Language**. The same word may mean different things in different bounded contexts, and that is acceptable because the meaning only has to be agreed within the context of a specific model. **Edges across which terms change meaning reveal the boundaries of the contexts.**

Example (Figure 4-1): the word "account" in an online reservation system:

Bounded context	Meaning of "account"
Identity and access management	A set of credentials used for authentication and authorization.
Customer management	A set of demographic and contact attributes.
Financial accounting	Payment information and a list of past transactions.

Not every term that comes up makes it into the Ubiquitous Language: only concepts **core to the context's primary purpose** do. These core concepts can be discovered from the **JTBDs** (Job Stories, Chapter 3) for the bounded context. The book recommends using the **key nouns in well-written Job Stories** to find the vocabulary terms (Figure 4-2).

Context mapping

DDD does not describe a complex system with one model; it designs multiple independent models that coexist and usually communicate through published interface descriptions. The representation of the domains in a larger system and how they collaborate is the **context map**; identifying and describing these collaborations is **context mapping**.

Relationship	Description
Shared kernel	The most basic type. Two domains developed largely independently end up overlapping on a subset of each other's domains, and agree to collaborate on it (possibly sharing code, data model and domain description). Problematic for microservices: needs high coordination to start and for every later change, creating many tight coordination points. If unavoidable, designate one team as primary owner/curator and the others as contributors.
Upstream–Downstream	The Upstream provides a capability; the Downstream consumes it. Domain definitions and implementations do not overlap, so this is more loosely coupled than a shared kernel. It appears in several forms (below).

Forms of Upstream–Downstream relationship

Form	Description
Customer–Supplier	Upstream (supplier) provides functionality to Downstream (customer). Upstream carries the overhead of backward compatibility; Downstream carries the risk of Upstream breaking something or ignoring its future needs.
Conformist	An extreme case: Upstream explicitly does not or cannot care about Downstream's needs ("use at your own risk"). Downstream must constantly conform to Upstream changes. Common when a large subsystem is used by a small one (e.g., a small new capability needing an enterprise payments system). Alternative: go separate ways, e.g., use a commercial payments vendor if the company allows.
Anti-corruption layer (ACL)	Downstream builds a translation layer between its Ubiquitous Language and the Upstream's, guarding itself from breaking changes. Effective and sometimes necessary, but can be expensive to maintain long term.
Open host service	When Upstream knows many Downstreams use it, it defines and publishes a standard interface that all consumers adopt, maintaining backward compatibility or clear versioning. Scales without drama. Practically all public APIs work this way (e.g., public cloud provider APIs).

Synchronous versus asynchronous integrations

Integration between bounded contexts can be either; no integration pattern fundamentally assumes one or the other.

- **Synchronous:** RESTful APIs over HTTP, gRPC services using binary formats such as protobuf, and more recently GraphQL interfaces.
- **Asynchronous: publish–subscribe** leads. Upstream generates events and Downstream workers process those they care about. It is more complex to implement and debug, but offers superior **scalability, resilience and flexibility**: multiple receivers, even on heterogeneous tech stacks, can subscribe to the same events in a uniform way.

DDD aggregate

An **aggregate** is a collection of related domain objects viewed as a **single unit by external consumers**. Consumers reference only one entity in it, the **aggregate root**. Aggregates hide internal complexity and expose only the "interesting" information and capabilities. In Upstream–Downstream mappings, the Downstream sees the Upstream as an aggregate or a collection of aggregates.

4.3 Introduction to Event Storming

DDD analyzes systems at the whole-system level (**"strategic"**) and in depth (**"tactical"**) and helps find fairly autonomous, loosely coupled subcomponents. In the early days of microservices DDD was widely proclaimed as the one true way to size services, but it was "much talked about, less practiced". The book says that as a sizing tool it was more hype than reality, for two reasons:

- **It is complex and expensive:** it requires a lot of knowledge and experience (Evans' book is about 520 pages, and more reading and project experience are needed), and there were not enough skilled people.
- **It is a time-consuming team sport:** you must get business, product and design teams to join long, intense domain-design sessions, which is hard to sell.

Event Storming was proposed by **Alberto Brandolini** as a fun, lightweight, inexpensive process heavily inspired by DDD. It finds bounded contexts in **hours instead of weeks or months**. It is not a full replacement for DDD and does not give all its benefits, but for discovering bounded contexts it is a good approximation.

> **KEY DECISION: Use Event Storming instead of formal DDD**
>
> Use the more lightweight Event Storming process instead of formal DDD to discover the main aggregates in your subdomain and identify the edges of the bounded contexts in your system.

Requirements for a session

- A very long, unobstructed wall (longer is better) and a long roll of paper (e.g., IKEA Mala Drawing Paper) taped to it with 1/2-inch white artist tape, with multiple "lanes".
- Many sticky notes in different colors, most importantly **orange and blue**; a Sharpie for every participant.
- About **four to five hours** from well-represented team members. Virtual sessions with digital collaboration tools are possible, but physical is preferred.
- **Broad participation is critical**: subject-matter experts, product owners, interaction designers and business stakeholders, not only engineers. A session limited to software engineers happens in a bubble and is mostly useless, since it cannot produce the cross-functional conversations needed.

The Event Storming process

Phase	Time (4-hour session)	What happens
1. Discover domain events	~30 min (30 to 60 min)	Each participant writes key domain events on orange stickies: one event per note, as a past-tense verb, placed on the timeline (earlier events left, later events right). No coordination between participants and no weeding out duplicates yet. Aim for at least 100 event notes.
2. Enforce the timeline	~45 min (30 to 60 min)	With the facilitator, arrange notes into a coherent timeline and remove duplicates; ideally walk through a "storyline", like a user journey. Questions

Phase	Time (4-hour session)	What happens
		and confusion are not resolved on the spot but captured as hotspots (usually purple stickies) to answer offline.
3. Reverse narrative and command identification	~60 min	Walk the timeline backward from end to start and identify the commands (things that caused the events) on blue stickies. Many commands map 1:1 to an event; some do not (e.g., "Submit payment authorization" triggers many events). Do not judge or tidy; the real world is messy.
4. Identify aggregates / bounded contexts	~30 min (15 to 25 min of rearranging)	Commands do not produce events directly; aggregates accept commands and produce events. Rearrange (breaking the timeline if needed) so commands going to the same aggregate, and the events it fires, are grouped around it. The wall becomes clusters of events and commands around aggregates. These clusters are the bounded contexts.
5. Competitive analysis	~15 min	Prioritize the bounded contexts (see below).

The times do not add up to four hours because of breaks and the facilitator's time to prepare the space and give initial guidance. Facilitation takes an experienced person, but it does not require DDD-expert effort.

Competitive analysis (prioritizing contexts)

Create a matrix of the bounded contexts and rank each on two properties, **difficulty** (effort) and **competitive edge**, using T-shirt sizes (S, M, L). This phase is not in Brandolini's original process; it was proposed by **Greg Young** for prioritizing domains in DDD (similar to "root", "supportive" and "generic" in DDD).

Combination	Guidance
Large competitive advantage / large effort	Design and implement in-house and spend most time on these.
Small advantage / large effort	Buy.
Small advantage / small effort	Great assignments for trainees.
Other combinations	A coin toss; needs a judgment call.

4.4 Introducing the Universal Sizing Formula

Bounded contexts are a fantastic starting point, but **microservice boundaries are not synonymous with bounded contexts** from DDD or Event Storming. Boundaries are **not constant over time**: they evolve and tend toward **increasing granularity** as organizations and applications mature (Adrian Cockroft observed this repeatedly at Netflix).

> **Nobody gets microservice boundaries perfectly at the outset**
>
> Successful adopters start with a small number of services closely aligned with bounded contexts, and **split services when they hit coordination dependencies they need to eliminate**. Teams are not expected to get boundaries right out of the gate; boundaries evolve with a general direction of **increased granularity**.

It is typically **easier to split a service than to merge services back together** or to move a capability from one service to another. That is another reason to start coarse-grained and split once you know more about the domain and have enough complexity.

The Universal Sizing Formula

Three principles that work well together:

1. **Start with just a few microservices**, possibly using bounded contexts.
2. **Keep splitting** as your application and services grow, guided by the needs of **coordination avoidance**.
3. **Be on the right trajectory for decreasing coordination.** This is vastly more important than how "perfectly" services are sized at present.

4.5 Chapter 4 Summary

- Sizing microservices by lines of code or by function (e.g., serverless functions) is flawed; boundaries should follow business capabilities and aim to minimize coordination.
- DDD offers bounded contexts, Ubiquitous Language, context mapping and aggregates; Event Storming is a fast, cheap way to find bounded contexts.
- The Universal Sizing Formula: start with few services, keep splitting to avoid coordination, and focus on the trajectory.

5. Dealing with the Data

The chapter covers why microservices must "own their own data" and what that means for architecture. It discusses the key data management patterns: **delegates, data lakes, sagas, Event Sourcing and CQRS**, illustrated with the sample project.

Data is one of the first challenges almost everyone hits. Logical and physical model design for microservices differs from designing tables for conventional **N-tier monolithic applications**. If it were not for data management challenges, breaking monoliths into loosely coupled, "bite-sized" microservices would be fairly easy.

5.1 Independent Deployability and Data Sharing

Following Sam Newman, microservices should be **loosely coupled** (which also means **independently deployable**) and **highly cohesive**. When services are loosely coupled, a change in one should not require a change in another. The main benefit of microservices (speed in harmony with safety and quality, at scale) comes from decreasing coordination needs.

> **Independent deployability**
> Being able to make a change to one microservice and **deploy it without changing or deploying any other microservice**. If deploying one service triggers ripple effects that force redeployment of others, the deployment pipeline (Figure 5-1: pipelines going through several environments to production) becomes complex and fragile, compromising both speed and safety.

Many things can prevent independent deployability, but in data management the most common offender is **co-ownership of a data space by multiple microservices**.

5.2 Microservices Embed Their Data

Monoliths and SOA: in monoliths, and even modular or service-oriented architectures (SOA), components co-own data as regular practice; shared data is the primary way of integrating modules. Monoliths do have modules, but those modules were **not independently deployable**, so they were not loosely coupled with respect to coordination costs. It was primarily **data coupling** that stopped SOA designs from achieving independent deployability, and so from being both fast and safe at scale.

> **Example of the problem**
> Several microservices (flight search, reservations, flight tracking) share a **customer table** and "ownership" means they read and modify it. If flight search changes a column type (e.g., integer to float), it can break reservations or flight tracking. To avoid bugs, the other services' code must change too, and all changed services must be **redeployed in one concerted effort**. These ripple effects couple the services.

Sharing data spaces is a primary killer of independent development and independent deployability. So in microservices, data sharing is prohibited: microservices are never allowed to co-own responsibility for a data space. It must be clear which microservice owns any dataset, summed up as: **microservices must own (embed) their data.**

Embedding data does not mean an explosion of database clusters

Datasets (e.g., relational tables) must never have multiple microservices as co-owners. But databases are complex systems, usually deployed on multiple servers for redundancy, reliability and scalability, possibly

dozens of servers across regions. Requiring a separate cluster (Cassandra, Postgres, Redis, MySQL, etc.) for each of hundreds of microservices would make microservices the most expensive style in history. The book says this is **not** required.

> **KEY DECISION: Microservices can share physical database clusters**
> Microservices can and should share physical installations of database clusters. As long as services never share the same logical table space and never modify the same data, sharing physical installations is OK in practice.

Data independence is about "not crossing the streams": being able to deploy your services with another database installation if needed. Cost and simplicity matter. As long as multiple services are not accessing (above all, **modifying**) the same data space, data independence holds.

Data embedding and the data delegate pattern

Modules in an N-tier monolith could be deployed as networked services and be small enough to be called "micro", but they are **microservices only if modularized with coordination elimination as the goal** (loosely coupled and independently deployable). Arbitrary splits that are not loosely coupled do not make a microservices architecture.

Scenario (Figure 5-2): three services (flight search, reservations management, flight tracking) all need the "flights" table. This shared data space compromises independent deployability.

Solution: hide the shared data behind a delegate service (Figure 5-3). The **flight inventory service** is declared the **authoritative service for all flight information**. Any service needing to read or update flight data must call an endpoint on it.

- If the flight lookup API is flexible enough, the old flight search service simply becomes part of the flight inventory service.
- Reservations (e.g., checking if enough seats remain) and flight tracking (e.g., reading or updating a plane's location) stop touching the flights table and use the inventory service instead.
- The delegate **hides, encapsulates and wraps around** the data. Alternatively, you can introduce a **new** service for flight inventory instead of converting an existing one, with flight search calling it like the others.

The delegate approach is elegant but cannot solve every case. Some functionality legitimately needs data across microservice boundaries (**analytics, data audit, machine learning**), and **traditional database transactions require locking shared data**.

Using data duplication: data lakes

For **read-only** access to distributed data (enterprise analytics, machine learning, audits), the common solution is to **copy** (not move) datasets from the microservices into a shared space, the **data lake**.

- Data lakes are **read-only, queryable data sinks** and **reference data stores**, never the database of record.
- Microservices remain the **authoritative sources** (systems of record, SOR) and primary owners. They stream relevant data into the lake, where it is indexed for query-ability.
- To keep data integrity and clear data lineage, data in the lake must **never be operationally updated**.
- Streaming is done through reliable messaging: IBM MQ and RabbitMQ have long been used; **Kafka** is currently most popular; **Apache Pulsar** is a prominent new entrant (Figure 5-4).

Distributed transactions and surviving failures

Example: booking a seat requires a coordinated update across microservices: payment (using loyalty miles), reservations (securing the seat) and notifications (sending an itinerary). We want **all three or none**. If the seat becomes unavailable while miles are being deducted (a race condition), the entire process must be rolled back and the miles refunded. In a busy system such failures are inevitable.

ACID transactions let monoliths do this safely, even with failures:

Property	Meaning
Atomicity	Steps are "all or nothing": either all execute or none.
Consistency	A transaction moves the system from one valid state to another valid state.
Isolation	Parallel transactions produce the same state as if executed sequentially.
Durability	Once committed, data is not lost despite any failures.

Failures cannot be prevented in any complex system, whether microservices or not; they must be accounted for with auto-recovery. But **ACID transactions are not viable across independently deployed microservices**, because they rely on exclusive locks, which would be impossible or very expensive when services embed their data and may not manipulate other services' data. Patterns better suited to distributed architectures are needed.

Sagas

Sagas were first described by **Hector Garcia-Molina in 1987** and popularized for distributed systems by **Clement Vasters's 2012 blog post**. In a saga, every step of the transaction performs its action **and defines a compensating action** to run if a later failure forces a rollback. A pointer to the compensating action (e.g., discovery information on a queue) is registered on a **routing slip** passed to the next step. If a later step fails, it triggers all the compensating actions on the routing slip, "undoing" the changes and bringing the system to a **reasonably compensated state**.

> **Sagas are not directly equivalent to ACID transactions**
> A saga does **not** promise the system returns to its initial state on rollback; it reaches a **reasonable state** reflecting an acceptable level of undoing.

Seat reservation example (Figure 5-5): if booking fails, the reservation is cancelled and compensating actions for the previous steps (notification and payment) are invoked.

- **Payment:** the compensating action refunds the money. A refund may not process immediately, so the system does not return instantly to its initial state, and the two payment actions visibly cancel each other rather than leaving no trace.
- **Notification:** an email or text cannot be literally recalled, so the compensating action may be a new message saying to disregard the previous one. The customer sees two messages instead of none.

> **The sequence of events in a saga is meaningful**
> Order steps carefully. It usually pays off to **move steps that are harder to compensate toward the end**. If business rules allow, put the notification at the very end, so it is sent only after earlier steps succeeded, avoiding correction messages.

5.3 Event Sourcing and CQRS

Delegates, data lakes and sagas solve many data isolation challenges but not all. Eventually relational modeling itself falls short of allowing the desired isolation and loose coupling. A common case is creating a **"join" across datasets owned by different microservices**. Relational modeling is rooted in data normalization, data reuse and cross-referencing common elements, so it is fundamentally **biased toward data sharing**.

Event Sourcing

A data modeling approach that avoids data sharing and has become popular in microservices. One of the earliest mentions was **Martin Fowler's 2005 article**; **Greg Young's 2014 conference talk** revived its popularity. Event Sourcing is about **storing events (facts) rather than the state of domain objects**. Young: state is a transient first-level derivative of your facts.

A "fact" is the value of an event occurrence, e.g., "the price of an economy seat on the LAX–IAD flight increased by $200".

Real-life examples

- **Accounting journal:** a classic event store. Accountants record individual transactions, and the balance results from summing them. They do not record only the resulting balance.
- **Chess game record:** you record each move, not the board position after each move. The board state is the sum of all moves. The book shows the first seven moves of Game 6, Kasparov versus IBM's Deep Blue (1997), and the position can be completely re-created from the move log.

Event Sourcing versus relational modeling

	Relational (state-oriented)	Event Sourcing
What is stored	The current state (e.g., current price of a seat).	Facts: incremental changes (events). State is calculated from the sequence of events.
Design effort	Many opinionated decisions about entity types, tables and relationships.	Uniform; mostly describing domain-relevant event fields from business logic. Less subjective table-shape and relationship crafting.
Relationships	Referential relationships (e.g., customer to accounts to payment methods to reservations and trips).	No referential relationships between entities.
Data sharing	Biased toward sharing.	Easy to segregate: each event type can be owned by a different microservice (e.g., a customer demographics service owns the "customer info entered" event).

In the book's customer example (Figures 5-7 and 5-8), the same state is reached either via related tables or via a sequence of events: contact information collected, personal account opened, payment method entered, reservations and completed trips, then a business account opened with its payment information, plus preference changes. The event view also shows the **exact sequence of facts** that led to the state.

What does an event look like?

An event needs only three parts: a **unique identifier** (e.g., a UUID, helpful in distributed systems), an **event type**, and the **data** relevant to that type. The current state is a function of the events.

```
{
  "eventId" : "afb2d89d-2789-451f-857d-80442c8cd9a1",
  "eventType" : "priceIncreased",
  "data" : {
    "amount" : 120.99,
    "currency" : "USD"
  }
}
```

Calculating current state with projections

A **projection** gives the state at a point in time from events. It uses a **projection function** that takes the current state and a new event and produces the new state. For example, a `priceUp` function increases the price by the event's amount (equivalent to an `UPDATE prices SET price=...` in SQL). Running a projection means applying the projection functions for all relevant events. The current state is the **left fold** of the events so far.

```
function priceUp(state, event) {
  state.increasePrice(event.amount)
}
function priceDown(state, event) {
  state.decreasePrice(event.amount)
}
let price = priceUp(priceUp(priceDown(s,e),e),e);
```

A key benefit: you can compute the state **as of any point in time**, enabling sophisticated analytics ("what was the state on a past date?").

Improving performance with rolling snapshots

Projections can be computationally expensive when a value results from thousands of changes (e.g., a bank balance). Instead of recalculating from the beginning, save **intermediary values** and compute from the last snapshot. Saved projections are usually called **rolling snapshots**. Choose snapshot moments using **natural time points in the domain**: e.g., banks snapshot balances at the end of month, quarter and year ("closing the books"), so the balance for January 15, 2020 needs only two weeks of calculation from the December 31, 2019 snapshot.

The event store

Event stores can be simple. They can be built on files, Amazon S3 buckets or any database that reliably stores a sequence of entries. The interface must support three functions:

1. Store new events and assign the correct **sequence**, so they can be retrieved in saved order.
2. **Notify subscribers** building projections about new events they care about, and support the **Competing Consumers** pattern.
3. Get **N events after event X** for a given event type, for **reconciliation** (recalculating if a projection is lost, compromised or doubted).

At its essence the interface is just two functions: `save(x)` and `getNAfterX()`, plus a robust notification system.

Competing Consumers: projection builders typically run multiple listener instances for redundancy and scalability, so the notifier must deliver each event **only once to a single instance**, avoiding duplication that could corrupt data. Two approaches:

- Use a message queue that already guarantees this to consumers, such as **Apache Kafka**.

- Let consumers register **HTTP callback endpoints**; invoke the callback for each new event and let a consumer-side **load balancer** distribute work.

Neither is inherently better (one is push-based, the other pull-based). The authors also published a skeleton reference event store implementation on GitHub.

Command Query Responsibility Segregation (CQRS)

Advanced event-sourced systems typically build projections with **CQRS**. Its idea: **the way we query systems and the way we store data do not have to be the same.**

The simple event store interface cannot run elaborate queries, e.g., "all reservations in which a passenger updated their seat in the last 24 hours". Such queries are not run against the event store, which stays simple and focused solely on **authoritatively and reliably storing the event log**. Instead, another system subscribed to the event store is told of each event and builds **indices optimized for querying**. The principle: do not solve **data storage, data ownership and data queryability** with the same system; solve them independently.

Win: Event Sourcing plus CQRS lets you design very granular, loosely coupled components, even microservices so tiny they manage one type of event or run a single report. Targeted use takes microservices to the next level of autonomous granularity.

> **Do not abuse Event Sourcing and CQRS as a cure-all**
>
> Use them **only when necessary**, since they complicate implementation. They should not be the sole data modeling approach for the whole system; the conventional relational model is often much simpler. They help avoid data sharing in sophisticated cases needing **joins across service boundaries**, but at a cost of complexity. **Consider simpler approaches such as the delegate service first.**

5.4 Event Sourcing and CQRS Beyond Microservices

Their benefit is not limited to loose coupling or microservices; they are powerful data modeling tools for many systems. Two major benefits:

1. Working with the CAP theorem

The **CAP theorem** was formulated as a conjecture by **Eric Brewer** in his 2000 keynote at the Symposium on Principles of Distributed Computing. In its original form, a distributed shared-data system can have only two of three properties:

Property	Meaning
Consistency	A single view of the latest state of the data.
Availability	Ability to always read or update data.
Partition tolerance	Getting accurate data even in the face of network partitions.

It was later clarified that not all combinations are valid. Network partitions cannot be avoided in distributed systems, so the real choice is a **sacrifice between consistency and availability**.

The catch: CAP constrains a *single* system with data sharing. With CQRS, we use **multiple systems and minimize data sharing**: prioritize **consistency in the event store** and **availability in the query indices**. The indices may get consistency wrong, but they are **not authoritative**, and can always be **re-indexed from the event store**. This gives "the best of both worlds".

2. Auditability

With a relational model, **in-place updates** (e.g., correcting a customer's address) may lose the history, leaving you helpless if the customer later disputes the record. With Event Sourcing, a **perfect history of every change** is preserved, including the value at any past time and how and when it changed.

Event Sourcing versus logging

Logging changes to a file or to tools like Splunk or ELK does not equal Event Sourcing. The difference is **which system is the source of truth**: in Event Sourcing the state is calculated from the events, so the event log is trusted. With Splunk logs, the source of truth is most likely the relational model, with logs used only occasionally to hunt bugs. **When your reliable log of events is your source of truth, you are using Event Sourcing; otherwise you are not, however many logs you generate.**

5.5 Chapter 5 Summary

- Data isolation and **embedding data in the owning microservice** is fundamental for loose coupling and independent deployability.
- Conventional (monolith-oriented) data modeling struggles with this; the toolset is **delegates, data lakes, sagas, Event Sourcing and CQRS**.
- Event Sourcing is a new data modeling approach, distinctly different from relational modeling, useful even beyond microservices.

Quick Revision Sheet

Key terms

Term	Meaning (as used in the textbook)
Bounded context	The range of applicability of a domain model (Evans); a starting point for service boundaries.
Ubiquitous Language	The shared vocabulary of a bounded context, developed by experts and engineers together; terms can change meaning across contexts.
Context map	Representation of the domains in a system and how they collaborate.
Shared kernel	Two domains overlapping and collaborating on shared code/data/model; high coordination, problematic for microservices.
Upstream–Downstream	Provider–consumer relationship: customer–supplier, conformist, anti-corruption layer, open host service.
Aggregate / aggregate root	Collection of related domain objects treated as one unit, referenced externally via the root entity.
Event Storming	Brandolini's lightweight workshop to find bounded contexts in hours.
Hotspot	Purple sticky note capturing an open question during Event Storming.
Universal Sizing Formula	Start with few services; keep splitting guided by coordination avoidance; stay on a trajectory of decreasing coordination.
Independent deployability	Ability to change and deploy one microservice without changing or deploying others.
Delegate service	A service that owns and hides shared data behind its API.
Data lake	Read-only, copied, queryable aggregate of data from many services; never a database of record.
Saga	Distributed transaction in which each step has a compensating action; yields a reasonable, not identical, state on rollback.
Routing slip	Carries pointers to compensating actions from step to step in a saga.
ACID	Atomicity, consistency, isolation, durability.
Event Sourcing	Storing events (facts) rather than state; state is derived.
Projection / rolling snapshot	State computed from events via a projection function / saved intermediate states for speed.
Event store	Stores events in order, notifies subscribers, and returns N events after X; interface: save(x), getNAfterX().
Competing Consumers	Pattern ensuring each event is delivered once to a single listener instance.
CQRS	Command Query Responsibility Segregation: querying and storing need not use the same system.
CAP theorem	Consistency, availability, partition tolerance: a distributed shared-data system cannot have all three.

Key decisions in Chapters 4 and 5

#	Decision / guidance
1	Use Event Storming instead of formal DDD to find aggregates and bounded contexts.
2	Avoid creating too many microservices too early; start with a handful and split over time.
3	Apply the Universal Sizing Formula: few services, keep splitting to avoid coordination, right trajectory.
4	Microservices must own (embed) their data; never co-own a data space.
5	Microservices can share physical database clusters, as long as logical table spaces and modified data are not shared.
6	Use a delegate service when several services need the same data.
7	Use data lakes (copied, read-only) for analytics, ML and audit needs.
8	Use sagas with compensating actions instead of ACID transactions across services; order steps so hard-to-compensate ones come last.
9	Use Event Sourcing and CQRS only when necessary, after considering simpler options such as delegates.